Lab 3 – Component Diagram: Written Justification

Prashanthi Kusupati | PES1UG24AM146 | System: LifeLine Emergency Blood Bank

1. Scenario and requirements

The system helps a blood bank manage donors and blood stock, with the emergency blood request as its most important use case. A hospital raises a request, the system checks stock, finds compatible blood groups, locates eligible donors if stock is short, and sends alerts. Functional needs: login by role, request handling, stock tracking, donor lookup and notifications. Non-functional needs: fast response in emergencies, simple screens for staff under pressure, and protection of donor and patient data.

2. Components and interfaces

Component	Responsibility	Interfaces
Portal UI	Screens for donors, hospitals and staff to log in and raise or track requests	Requires IAccess, IRequest
Emergency Request Coordinator	Central coordinator: receives an emergency request and drives the other services	Provides IRequest; requires IAccess, IDonorLookup, INotify, IMatch, IStock, IRequestStore
Access Control Service	Logs users in and checks their role (donor, hospital, staff)	Provides IAccess; requires IAccessStore
Donor Registry Service	Finds registered donors who are eligible and nearby	Provides IDonorLookup; requires IDonorStore
Notification Dispatcher	Sends alerts to donors and status updates to hospitals	Provides INotify; requires ISmsGateway
Blood Group Matcher	Applies blood group compatibility rules to a request	Provides IMatch
Stock Ledger Service	Tracks blood units and expiry, checks and reserves stock	Provides IStock; requires IStockStore
Blood Bank Database	Stores users, donors, stock and requests	Provides IAccessStore, IRequestStore, IStockStore, IDonorStore

3. Why I designed it this way

- **Coordinator as hub:** the UI only calls IRequest. Matching, stock, donors and alerts can change without touching the screens.
- **Provided/required interfaces:** every dependency is a ball-and-socket interface, so a component can be replaced if it keeps the same interface, for example another SMS provider.
- **Separate Matcher and Stock Ledger:** compatibility rules and stock counts change for different reasons, so they are kept apart and easy to test.
- **Notification Dispatcher isolates the external gateway:** only this component knows about SMS or email, so other services never depend on it.
- **Database behind four store interfaces:** each service reaches only its own data, which keeps the data layer loosely coupled.

4. How the main challenges are handled

Performance: stock and match checks are small, separate calls, so an emergency request is answered quickly. **Usability:** the Portal UI only handles display and input. **Security:** every request goes through IAccess, and sensitive records are reachable only through the owning component's store interface.

5. Data flow (emergency request)

Portal UI raises the request through IRequest → Coordinator verifies the user (IAccess) → asks the Matcher for compatible groups (IMatch) and the Stock Ledger for units (IStock) → if stock is short, queries donors (IDonorLookup) → sends alerts through INotify, which uses ISmsGateway → saves the request (IRequestStore) and returns the status to the UI.